SOURCE REVIEW AND CHECKED EXAMPLES

Shorter Lean proofs for DiophantineConsistency

Reusable proof interfaces, smaller generated scripts, and a practical adoption order

Prepared for Vladimir Reshetnikov 26 October 2026

Abstract

The largest opportunity in this development is to centralize recurring proof arguments while keeping the arithmetic data and public theorem statements stable. The repository already contains strong abstractions: topological circuit semantics, a power certificate, structural degree bounds, assignment renaming, and a semantic window interface. Several clients still repeat the arguments that these interfaces can discharge.

This report identifies concrete sites in the pinned source, supplies independently checkable Lean examples and reviewable patches, and separates completed compilation checks from source-level proposals. A generated-circuit candidate reduces two files from 6,567 to 4,917 physical lines, including its new helper and certificate data, while preserving 1,192 public theorem signatures. That is a measured source reduction, not a claim of a faster build or a completed integration. Smaller primitive-recursion replacements have been checked in complete modified files.

Recommended first changes. Adopt the checked recursion-congruence proofs; replace private list-lookup inductions with existing library facts; expose one assignment-transport lemma; consolidate sums of squares; then test the generated circuit certificates. Follow with scanner invariants, digit/carry interfaces, and the model-theoretic bridges.

Scope. Source snapshot `60e9c5d620c6c387d932876cfa2d9376f64ea56c`; Lean `v4.32.0`; mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`. The report concerns proof engineering. It does not assert a new equation, a reduced number of variables, a proof of consistency, or completion of an open equivalence link.

Deliverables. The archive contains this PDF and its standalone LaTeX source; Lean demonstrations; patches and generated candidate files; reproducible census and static-audit scripts; hashes, measurements, and compilation receipts. The validation table is the authoritative guide to which artifacts were actually compiled.

Contents

1 What was reviewed and what the measurements mean 3

2 Priority map 3

3 Small proof replacements with immediate value 4

4 Sums of squares: expose the common mathematical argument 5

5 The largest measured source reduction: generated circuit proofs 6

6 A shared induction for topological circuit semantics 8

7 Translation certificates, structural degrees, and register extension 8

8 Scanner proofs and numeric Boolean interfaces 9

9 Digit, carry, and phase proofs 10

10 Object-language proofs: remove administration, retain definability 11

11 Automation and performance: a measured workflow 12

12 Validation status and reproduction 13

13 Further questions worth investigating 14

Source and artifact guide 15

1 What was reviewed and what the measurements mean

The maintained `Computability/DiophantineConsistency/Lean` tree contains 546 Lean files and 73,971 physical lines at the selected commit. There are 363 files in its namespace directory, including embedded check modules, 182 files in the separate `checks` directory, and a 283-line umbrella import file. A lexical census finds 59,832 nonblank code lines and 4,691 theorem/lemma declarations after masking comments and strings. These are source measurements, not counts of elaborated declarations or proof terms.

The wider requested directory also contains 165 loose Lean files outside that maintained tree, largely in archived proposals and provenance copies. They were inventoried separately. Rewriting such snapshots would not shorten the maintained build and could invalidate their historical meaning. The recommendations therefore target the maintained modules. The full path list, per-file hashes, declaration index and counting script are included.

Source feature	Lexical count
`omega` occurrences	1,681
`nlinarith` occurrences	563
`ring` occurrences	686
`simp_all` occurrences	63
`maxHeartbeats` settings	134
`maxRecDepth` settings	177

These counts locate repeated patterns; they do not establish that a tactic is slow. A large heartbeat allowance is a place to investigate, not a measured performance bottleneck. The masked source scan finds no `sorry`, `admit`, `native_decide`, or `axiom` tokens in the maintained tree. That observation is strictly weaker than a transitive axiom audit. The new checked examples include `#printaxioms` output so that their actual dependencies can be inspected.

The source pin and build versions come from the repository's toolchain file and dependency manifest. The equivalence plan distinguishes existing external Lean endpoints, the remaining literal arithmetic/native links, and the additional work needed for an internal ZFC assertion. Refactoring must preserve those distinctions.

2 Priority map

The priorities below balance reviewability, reuse, and the chance of reducing future work. Numerical savings are given only when an actual candidate was measured. More abstract interfaces have intentionally unquantified savings until their clients are migrated.

Order	Change	Benefit and adoption condition
1	Recursor congruence	34 lines removed in three compiled modified modules; executable constructor data unchanged.
2	Core list-lookup lemmas	40 lines removed from seven private wrappers; exact replacement signatures checked; full client rebuild remains.
3	Assignment transport	One naturality lemma replaces repeated extensionality; candidate saves 25 lines including the helper.
4	Shared sum-of-squares criterion	Makes the arithmetic reason for zero explicit and centralizes syntax evaluation.
5	Generated step and power certificates	Candidate saves 1,650 lines across two files; preserve signatures and update source generators before integration.

Order	Change	Benefit and adoption condition
6	Generic topological invariant	Shares dependency-bound induction for equality, nonnegativity and degree.
7	Recursor-level scanner laws	Reuses fixed-point, first-step and invariant lemmas without changing the scanner code.
8	Conditional padding witness	Require the witness only in the phase where it is used; remove the dummy branch.
9	Digit/carry normalization interface	Derive quotients and modular projections once; retain strict digit bounds and cyclic semantics.
10	Commutative circuit relation	Absorb swapped multiplication rows into the finite certificate; subtraction remains ordered.
11	Early syntax-degree module	Reuse the existing theorem without import cycles; preserve cancellation identities.
12	Definability and sealed-model bridges	Share model-theoretic administration while retaining actual formula witnesses and all models.

3 Small proof replacements with immediate value

3.1 Rewrite under primitive recursion

The `Realizes.prec` constructors in `ScalarPRCode.lean`, lines 64–80 and `PRCode.lean`, lines 88–104 already receive pointwise correctness equations for the base and step programs. Their existing proofs introduce another induction merely to transport those equations through `Nat.rec`. Simplification can perform that transport directly, including under the step lambda.

After the unchanged constructor and argument introduction, the scalar correctness proof becomes:

```
simp only [eval, Nat.unpaired, p.property, q.property]
```

The vector version uses `[eval,p.property,q.property]`. The analogous recursion branch in `ScalarPROptimizer.lean`, lines 119–133 becomes:

```
| prec f g hf hg =>
    simp only [optimize, eval_smartPrec, eval, hf, hg]
```

The three fragments shrink from 12, 11, and 15 lines to 1, 1, and 2 lines: 34 lines saved. Complete copies of all three modified files were compiled. `PRProofChecks.lean` additionally proves by `rfl` that the replacement constructors return the same executable `.val` programs. This matters because these constructors carry programs as data: a shorter semantic proof should not silently change the source program whose size is measured elsewhere.

This pattern generalizes. Before writing an induction that compares two recursors, ask whether their base and step functions are already equal pointwise. Use an explicit simplifier list to expose that congruence. Retain real inductive arguments when the invariant depends on the recursion index or on an accumulated semantic property. Rewriting under a lambda can use function extensionality, so the exact standard-axiom footprint need not equal that of the original induction. If preserving that smaller footprint matters, the included axiom-free `RecursorCore.rec_congr` helper proves the transport once by induction and can be reused instead.

3.2 Use the core list-lookup API

Five private helpers prove that successful optional lookup implies list membership; two helpers prove the converse existential lookup statement. The pinned Lean core already has the exact facts:

```
List.mem_of_getElem? h
List.getElem?_of_mem h
```

The sites are AnnotatedProofChecker.lean, lines 177–187, AnnotatedRename.lean, lines 110–117, AnnotatedBound.lean, lines 50–57, AnnotatedRuleSide.lean, lines 18–35, and AnnotatedAssumption.lean, lines 40–57. Keeping the private wrapper names makes this a small integration change: replace each body, preserve its statement, and leave callers alone.

The supplied patch reduces the seven declarations from 61 to 21 physical lines. The exact generic types were compiled in ListChecks.lean, and the patch passes gitapply--check. This does not substitute for rebuilding those five modules and their downstream clients; that integration step is explicitly pending. The gain is that ordinary list facts have one maintained proof upstream instead of seven local inductions.

3.3 Expose assignment naturality once

Representation.lean, lines 155–231 repeatedly proves function extensionality to show that renaming variables commutes with the assignment combining source variables and witnesses. Conjunction and disjunction each repeat a left and a right version. QuarticCompiler.lean, lines 95–98 already contains a narrower private assignment_comp lemma.

Generalize that idea at the earlier representation boundary:

```
theorem natAssignment_comp_map
    (v : sigma' -> Nat) (w : tau' -> Nat)
    (f : sigma -> sigma') (g : tau -> tau') :
    natAssignment v w ∘ Sum.map f g =
      natAssignment (v ∘ f) (w ∘ g) := by
  funext i
  cases i <;> rfl
```

Pair the lemma with Expr.eval_rename, and the client proofs can normalize assignments without constructing fresh extensionality proofs. The proposed patch saves 25 lines in the complete module, counting the new helper.

Start with explicit simponly uses. A new global simplification attribute is unnecessary for realizing this saving, and a deliberate normalization direction is easier to review. The executable expressions, witness counts, and coordinate injections should stay identical. The artifact table records both the generic lemma check and the complete-file patch check separately.

4 Sums of squares: expose the common mathematical argument

4.1 The five-residual theorem

ShortSystem.lean, lines 64–91 proves that a sum of five integer residual squares vanishes exactly when all five

residuals vanish. Its forward proof derives each squared residual separately with `nlinarith`, supplying the same nonnegativity facts repeatedly.

Write the residual vector as

$$r = (n_0 - 1,\ n_1 - 1,\ n_a - 1,\ n_s - 1,\ n_k n_t - 1).$$

Then use the already available `Finset.sum_mul_self_eq_zero_iff` and simplify the five-element sum. The checked `ResidualSumChecks.lean` states the same theorem against the original `ShortSystem.residualSum`; its body is a single specialization followed by simplification. The exact replacement compiled on Lean 4.32.0. The supplied patch also adds the required finite-sum imports; the full patched module has not been rebuilt.

This improves the proof's explanation: the conclusion follows from the zero criterion for a sum of squares over the integers. It also avoids five solver calls on a repeated nonlinear expression. No build-time improvement is asserted without measurement. The total ordered-ring assumptions are essential to the shared theorem; the statement cannot be transplanted unchanged to arbitrary commutative rings.

4.2 Share the syntax-level theorem across compilers

There is a second, more reusable opportunity. `DAGQuartic.lean`, lines 83–98, `NaturalDAGQuartic.lean`, lines 110–126, and `QuadraticRegisterSystem.lean`, lines 70–87 repeat a private evaluation lemma for `QuarticSyntax.sumExpressions`, then specialize the same sum-of-squares argument.

Place a common theorem near `QuarticSyntax` for an arbitrary residual family `Finn->Exprsigma`:

$$\mathrm{eval}\left(\sum_i r_i r_i\right)(v) = 0 \quad \Longleftrightarrow \quad \forall i,\ \mathrm{eval}(r_i)(v) = 0.$$

Each compiler's `expression_zero_iff` then becomes one application. The residual definitions continue to show which local equations are being imposed; the helper only centralizes how conjunction is represented by squares. `CircuitRefactoring.lean` supplies the common theorem and both signed-DAG and natural-DAG clients.

The same approach applies to cost proofs. `DAGQuartic.lean`, lines 166–178 has a general sum-cost bound, while `QuarticCost.lean`, lines 65–74 has a special bound of five operations per term. Move the more general bound to a lower layer and derive the specialized result. Avoid a dependency from basic syntax back into a particular compiler.

5 The largest measured source reduction: generated circuit proofs

5.1 Reuse the power certificate that already exists

`ArithmeticDAGPowers.lean`, lines 24–42 proves `run_power` from topological correctness and a finite exponent-label certificate. The generated `GapCircuitSteps66` and `SplitCircuitSteps59` files nevertheless contain 86 and 107 individually scripted pure-power proofs. Those proofs rewrite earlier facts and then invoke `ring`.

The proposed generator emits one exponent-label list, one kernel-checked closed `PowerStep` certificate, and one specialization of `run_power` per circuit. Every existing public `natural_power_*` theorem keeps its explicit type and becomes an application of that helper. A representative body changes from:

```
by
  rw [natural_step_25]
  simp only [natural_power_24]
  ring
```

to an application of the form:

```
natural_power p x ⟨25, by decide⟩ (by decide)
```

The complete helper, certificate, and generated clients are included in the candidate source tree.

The label zero means "no asserted power specification," so its nonzero-label premise must remain. Check the labels against the actual stored node list and multiplication references. A correct recurrence over an unlinked duplicate circuit would not prove anything about the circuit used by the final theorem.

5.2 Specialize local evaluation once per circuit

The two files also contain 484 and 515 explicit local step equations. Their proofs repeatedly instantiate `run_node_equation`, rewrite the same length and discharge a numeric index bound. One helper with an index `Fin484` or `Fin515` performs the circuit-specific work once. Each public equation can retain its useful concrete right-hand side and use that helper with a literal finite index.

The generated candidate combines these two changes. Source measurements include all new label data and helper declarations:

File	Old lines	New lines	Old nonblank	New nonblank
Gap Steps	3,122	2,354	2,378	1,775
Split Steps	3,445	2,563	2,607	1,932
Total	6,567	4,917	4,985	3,707

The measured saving is **1,650 physical lines (25.1%)** or 1,278 nonblank lines (25.6%). All 999 step theorem signatures and 193 power theorem signatures are retained byte for byte. The 193 `ring` calls disappear. The independent static checker also checks the literal multiplication references, nonzero power labels, and circuit topology.

5.3 Integration conditions

The generated source is a candidate, with its validation status recorded separately. Signature equality, topology checks, and arithmetic label checks do not prove that Lean elaboration succeeds. They also do not measure proof-term size or kernel time. A large closed `decide+kernel` certificate can be expensive even when its source is one line. If necessary, shard it by register ranges and combine the results with a small checked theorem.

Update the upstream `tools/generate_split_recipe_steps59.py` template and its gap wrapper before adopting the generated patch. Applying only generated output would fail later generator comparisons. The supplied generator intentionally writes a separate candidate tree and does not mutate the checkout. In the split family,

`SplitJointPower59` already carries related power-label data; extract common label data to an earlier module instead of importing a later recipe and creating a cycle.

After the candidate files compile, rebuild the corresponding Registers, Recipe and Translation modules. They use the named equations as rewrite rules, so retaining names and types is necessary but a downstream replay remains useful. Compare exact public theorem types, circuit data hashes, axiom closures and regeneration output. The operation count, arithmetic graph and polynomial should not change.

6 A shared induction for topological circuit semantics

Three proofs follow the same structure: `ArithmeticDAGSemantics.lean`, lines 97–109 for uniqueness, `ArithmeticDAGDegree.lean`, lines 65–82 for degree bounds, and `NaturalDAGQuartic.lean`, lines 43–53 for nonnegativity. Each performs strong induction on the register index, rewrites the local node equation, and uses topology to bound every dependency twice.

Factor out a theorem with an indexed property $P(i, z)$. Its hypothesis says that evaluating node i preserves P whenever every operand satisfies its corresponding property. The conclusion says that every valid entry of the run satisfies P. The proof contains the common topological induction. Its clients become:

- *Uniqueness:* $P(i, z)$ is $z = v_i$, and the local step uses node-evaluation congruence and the supplied equation for v_i.
- *Nonnegativity:* $P(i, z)$ is $0 \le z$, and the local step is the existing allowed-node lemma.
- *Degree:* $P(i, p)$ is totalDegree$(p) \le d_i$, and the local step is the existing degree-certificate lemma.

The artifact `CircuitRefactoring.lean` proposes this theorem and all three clients. Its isolated subset `CircuitInvariant.lean`, containing the generic invariant plus uniqueness and degree clients, compiled successfully. The nonnegativity extension and full SOS prototype remain source-only. The helper costs roughly one old induction; its saving is amortized across the clients. The more important benefit is that each client exposes its own mathematical invariant without repeating index management.

Do not force every circuit theorem into this interface. The designated base of `run_power` is itself defined by the actual evaluation, and a generic arbitrary-environment local step needs an additional seed mechanism to handle it. Reusing its existing specialized theorem is clearer than enlarging the invariant interface prematurely.

7 Translation certificates, structural degrees, and register extension

7.1 Treat commutative swaps in the relation

`GapCircuitTranslation66.lean`, lines 16–76 and `SplitCircuitTranslation59.lean`, lines 16–76 each exclude one row from the ordinary finite `NodeRel` certificate, then handle that row with a custom multiplication rewrite. The exceptional rows are 421 and 452. Their issue is operand order.

Extend the local relation so that addition and multiplication accept either the ordered related operand pair or the swapped pair. Prove evaluation preservation once, using `add_comm` and `mul_comm`. Subtraction must retain ordered operands. The exceptional row then participates in the same finite certificate as the other rows. `NodeRelCommutative.lean` supplies a candidate implementation.

The next layer is a shared translation theorem parameterized by the two node lists, input correspondence, reference map, output bounds and local relation certificate. Use the existing uniqueness theorem to derive evaluation equality.

A structure that merely stores the desired final equality would hide the repeated work instead of doing it once.

7.2 Move existing degree machinery to an earlier layer

`QuadraticRegisterSystem.lean`, lines 18–33 already defines `syntaxDegree` and proves a polynomial-degree upper bound. This is useful infrastructure in a module too late in the import graph for the earlier compilers. Move it to a small expression-degree module importing representation syntax and polynomial degree facts. Then routine wire, right-hand-side and residual bounds can reduce to finite syntax arithmetic.

This method gives an upper bound. It does not exploit cancellation, prove that the highest coefficient is nonzero, or establish exact degree. In particular, `ShortFrontier.degree_main_norm` first proves an algebraic normal form of lower degree. Preserve that identity and apply the structural bound to the proved normal form. Keep exact-degree lower bounds separate.

7.3 Package append invariants locally

`NaturalRegisterProgram.lean`, lines 72–98 repeats the old-index/new-last-index split for topology and allowed-node properties. The evaluation lemmas at lines 129–152 repeat list-append bookkeeping. Add generic append lemmas beside the DAG semantics, then instantiate them using the already existing bounded `Op` interface, `Op.dependency_lt`, and `Op.allowed`.

A wholesale switch from lists to arrays or a differently indexed evaluator would touch many proofs and definitions. It may eventually improve execution, but it should follow profiling. A few append lemmas capture the present proof simplification with a much smaller compatibility burden.

8 Scanner proofs and numeric Boolean interfaces

8.1 Share iteration laws without changing the programs

The same expression `fuel.recinitial(fun_acc=>stepacc)` appears in `AnnotatedRenameScan.lean`, lines 29–55, `AnnotatedBoundScan.lean`, lines 25–48, `AnnotatedLineScan.lean`, lines 32–57, and `AnnotatedContext.lean`, lines 91–119. Each proves its own variants of zero absorption, successor iteration and completed-state stability.

The checked `ScannerCore.lean` module supplies three facts at the recursor level:

- a fixed state stays fixed after any number of steps;
- one step can be moved into the initial state of an iteration;
- a preserved predicate holds after every finite iteration.

These proofs use only `Init`; their axiom audits are empty. An existing `scan_zero` can become an application of the fixed-state theorem. A completed-state theorem uses the same fact at its own terminal state. The executable scanner definitions can remain literally unchanged, protecting the PR programs and size bounds built from them.

For larger soundness proofs, define the one-step invariant explicitly. In `AnnotatedLineScan.scan_sound`, it must still include the absorbing zero state and a history of genuinely provable judgments. The generic preservation lemma cannot repair a weakened invariant. Simulations of complete encoded lists are a second stage: factor them only after exposing their exact list/code correspondence.

8.2 Replace acceptance equivalences by exact numeric values

`AnnotatedRenameLocal` and `AnnotatedBoundLocal` already establish that their numeric classifier is Boolean and that it equals one precisely when the row validator accepts. Their scanner clients reconstruct the rejecting value repeatedly; see `AnnotatedRenameScan.lean`, lines 57–69 and the parallel bound-scanner proof.

Combine the facts once:

$$(n = 0 \vee n = 1) \ \wedge \ (n = 1 \leftrightarrow b = \text{true}) \quad \Longrightarrow \quad n = \begin{cases} 1 & b = \text{true}, \\ 0 & b = \text{false}. \end{cases}$$

`BooleanCore.lean` checks the generic lemma. The scanner proof can then rewrite the numeric result directly before splitting on the Boolean value.

Both premises matter. The equivalence $n = 1 \leftrightarrow b = \text{true}$ alone says nothing about whether a rejected value is zero or two. Also retain the raw-natural Booleanity theorem: a result only on canonical encoded rows does not replace the raw-code soundness argument.

9 Digit, carry, and phase proofs

9.1 Derive weaker moduli from stronger ones

`CurrentClause2011.lean`, lines 122–130 proves a reflected-weight residue modulo 2048. `CurrentEarlyPadding2011.lean`, lines 99–107 repeats the same finite case analysis modulo 16. Since $16 \mid 2048$, use the existing modular implication and `Nat.ModEq.of_dvd`. Their private selector-unit definitions are definitionally the same.

This particular client can reuse its existing import directly; no new shared definition is required for the supplied patch. If more earlier geometry clients need the same fact, move the common residue theorem to a low-level module instead of importing the later clause layer and creating a cycle. Keep the coefficient and index conventions explicit; a congruence for a shifted or differently indexed coefficient is not interchangeable.

9.2 A half-radix shift needs arithmetic, not sixteen cases

The half-radix cases of `NativeCyclicRotation.shiftDigits` reduce to the exact identity, for $s > 0$,

$$(sa) \bmod (2s) + \left\lfloor \frac{sb}{2s} \right\rfloor = s(a \bmod 2) + \left\lfloor \frac{b}{2} \right\rfloor .$$

From $b \leq 3$, the result is at most $s + 1$, because $a \bmod 2 \leq 1$ and $b/2 \leq 1$. If s is even, its parity is exactly $b/2$. This removes the need to enumerate both current and predecessor digits. It even shows that the bound on the current digit is unnecessary for these scalar conclusions.

`HalfShiftArithmetic.lean` proves the identity, bound and parity result using only `Std.Tactic`; all three compile. The intended clients are `NativeCyclicRotation.lean`, lines 100–116 and lines 160–165. Applying them to the cyclic predecessor still requires the original positivity and indexing facts. The checked scalar theorem is not a completed replacement of those project clients.

9.3 Make the padding witness conditional

In `NativePreliminaryCount2011.total_le_one_all_phases`, the padding function is quantified outside the hypothesis saying that it is relevant only in middle phases. As a consequence, `CurrentCount2011.lean`, lines 46–54 constructs an artificial zero function in the other phases.

Change the interface from a globally supplied function with a conditional property to a conditional existence:

$$7 \le \text{phase} \longrightarrow \text{phase} \le 10 \longrightarrow \exists u,\ \text{PaddingFacts}(u).$$

Destruct the witness only inside the middle-phase branch. The outer caller can forward its real padding-existence theorem directly. The supplied two-file candidate saves ten physical lines and removes a dummy witness. `ConditionalPadding.lean` states an alternative theorem against the existing lower-level count lemmas.

This is an API change, unlike the proof-body-only patches. It needs all callers to be checked. Only one production caller was found at this snapshot. The underlying mathematical requirement is unchanged: the function is only needed when the middle-phase condition holds.

9.4 Use one exact decomposition interface

The repository already has `CurrentPolynomialWindow2011.Facts`; a recommendation simply to "bundle the window facts" would repeat existing work. A useful refinement is a small normalization view centered on

$$R = d + Bc, \qquad 0 \le d < B.$$

It gives $c = \lfloor R/B \rfloor$, and if $m \mid B$ it gives $d \bmod m = R \bmod m$. If $R < B$, the outgoing carry is zero. `NormalizationArithmetic.lean` checks these three scalar facts. They apply to repeated modular rewrites in `CurrentClause2011.lean`, lines 160–166, `CurrentEarlyPadding2011.lean`, lines 251–258, and `CurrentPhaseParity2011.lean`, lines 113–117.

The strict digit bound is needed for the quotient formula. The decomposition equation by itself allows many digit/carry pairs. The modular projection needs the divisibility premise, while the small-total reset can derive the necessary digit bound. This interface must retain the incoming carry as part of R and must not introduce a fictitious zero starting carry into a cyclic stream.

There is also duplicated extraction of canonical digit witnesses in `CurrentSparseDigits2011.lean`, lines 118–129 and `CurrentHighDigits2011.lean`, lines 141–152. A shared witness-producing lemma can return the length, digit family and its actual normalization facts once. That is preferable to asking every later argument to destruct the same existential representation independently.

10 Object-language proofs: remove administration, retain definability

10.1 A definable-induction transport lemma

The `Object*` modules repeatedly use `omega_ind` through an explicit formula and its satisfaction equivalence. Examples are `ObjectAdditionLaws.lean`, lines 52–72, lines 86–105 and 126–150, together with `ObjectMultiplicationLaws` and `ObjectDistributivity`. Their arithmetic induction steps are substantive; the repeated semantic transport can be shared.

A proposed `omega_ind_of_spec` should take an actual formula, environment, and a theorem identifying satisfaction of that formula with the desired predicate. It transports the base and step cases to `omega_ind`, then transports the result back. Each arithmetic proof can work with its named semantic relation while displaying the formula that witnesses definability.

This is an architectural proposal. Arbitrary induction on every external predicate over the model's naturals is too strong. Replacing it with induction on Lean's `Nat` is also inappropriate when the internal model may be nonstandard. The supplied specification of the defining formula is the essential boundary. `ObjectOmegaSemiring` already supplies a semiring interface; adding another semiring abstraction would miss the actual repeated work below it.

10.2 One bridge from model validity to a sealed derivation

Several endpoints repeat the same sequence: apply `completeness_inf_context`, obtain the model, build its arithmetic axioms with `axioms_of_zfc_model`, and invoke `seal_valid` before the local semantic argument. See `ObjectAdditionLaws.lean`, lines 183–206, `ObjectDistributivity.lean`, lines 132–156, and `ObjectMultiplication.lean`, lines 198–219. A model-extraction lemma is itself duplicated privately in `ObjectArithmetic` and publicly in `ObjectMultiplication`.

Move that extraction theorem earlier and add one semantic-validity-to-sealed-provability bridge. Its hypotheses must quantify over all the models and environments required by the existing completeness theorem, and its conclusion must be the same sealed formula. Callers can then provide just the arithmetic argument.

Keep endpoints with a nonempty context separate unless a context-aware bridge is proved. This refactor should make the exact assumptions easier to see. It does not establish a missing arithmetic/compiler connection or create a new internal consistency result by changing the presentation of the endpoint.

11 Automation and performance: a measured workflow

11.1 Prefer a stable simplification boundary

The most maintainable short proofs usually unfold one semantic layer and use named facts for the next. Keep executable code, expression evaluation, arithmetic normal forms and model interpretation as separate simplification layers. A proof which unfolds every implementation at once may be shorter in source while producing larger goals and coupling distant modules.

Use `simp?` during development to discover a sufficient explicit set; retain `simponly` where stable dependencies matter. The Lean manual documents both the explicit set and ordinary simplifier behavior.[1] Do not mechanically replace every `simp` with a long explicit list. Small, conventional algebraic normalizations are often clearer with the ordinary simplifier.

Use `omega` for bounded natural/integer arithmetic after the mathematical structure is exposed. Use `ring` for algebraic identities, `nlinarith` where inequalities genuinely need nonlinear combination, and `decide+kernel` for closed finite certificates when appropriate. Replacing all proofs by broad automation would obscure the very interfaces this report recommends.

11.2 Measure four distinct things

For a proposed refactor, record source size, elaboration/kernel-checking time, proof-object size if relevant, and dependency impact. They can move in different directions. A 1,650-line source reduction says nothing by itself about a 25% time saving. Similarly, a larger generic theorem can make dozens of client proofs clearer without reducing the first migrated file.

Profile a representative old and new theorem under the same toolchain, imports and cache state. Separate cold dependency builds from repeated theorem elaboration. Use a wall-clock command around Lean and, when needed, `set_optiontrace.profilertrue` locally to identify which phase consumes time. Retain the underlying command and output, rather than inferring performance from heartbeat limits or one noisy parallel run.

[1] `https://lean-lang.org/doc/reference/latest/The-Simplifier/Invoking-the-Simplifier/`; accessed 26 October 2026. Exact examples in this packet were checked against Lean 4.32.0.

11.3 Keep the proof trust visible

The new examples use ordinary proof-producing tactics and include axiom reports. A source reduction should not obtain its apparent simplicity by adding `sorry`, assuming the desired correctness theorem, or substituting native computation for the existing trust policy. The official Lean manual explains that native evaluation introduces an additional trust dependency beyond kernel proof checking.[2]

Do not turn finite data verification into an unlinked Python assertion. Python is useful for generating and auditing labels; a Lean theorem must still bind those labels to the actual circuit data. Conversely, the static scripts in this archive are deliberately labelled as static checks, not replacements for Lean.

12 Validation status and reproduction

Artifact	Validation scope
PR congruence patch	Three complete modified files compiled; separate program-value equality checks passed.
List lookup replacements	Exact generic helper types compiled; five-file patch applies cleanly; complete client rebuild pending.
Assignment transport	Generic lemma and complete modified Representation module compiled; 25 net lines saved.
Scanner and Boolean helpers	Core-only Lean examples compiled; full scanner migrations are proposals.
Digit/carry scalar helpers	Six arithmetic theorems compiled; integration into current window modules remains a proposal.
Circuit candidate	All 1,192 signatures and literal power labels statically checked; full Lean integration not run.
Circuit invariant	Generic invariant and same-statement uniqueness/degree clients compiled; nonnegativity extension remains proposed.
Five-residual sum of squares	Same-statement replacement compiled against the original definition; full patched module not rebuilt.
Remaining integration proposals	Conditional padding, modulus reuse, full compiler SOS and commutative node relation are source-only.

Ten demonstration files and four complete modified source modules passed the recorded checks. The scalar and vector recursor replacements add standard extensionality dependencies relative to their original proofs: the scalar changes from no axioms to `propext,Quot.sound`; the vector adds `Quot.sound` to its existing `propext`. The optimizer's footprint is unchanged. The separate `rec_congr` alternative is axiom-free, but its caller adaptation is not part of the checked patch.

The examples are separate namespace additions and proof replacements prepared outside the original source tree. There was no remote commit or pull request. The compilation receipts cover the listed examples and complete modified files only; they are not an all-module build of the 546-file subtree. The source-only candidate files are clearly labelled.

12.1 Reproduce the source checks

Use the pinned commit and toolchain in an isolated checkout. The packaged scripts accept explicit paths. Run the census against a full checkout to reproduce the additional 165 archived Lean paths as well as the maintained-tree counts. The generated-circuit auditor compares the public statements byte for byte and independently checks

[2] `https://lean-lang.org/doc/reference/latest/Axioms/`. The packet's actual `#printaxioms` output, on the pinned version, is the relevant evidence for each checked example.

the literal circuit label conditions. The report's original source metrics are also supplied so that the baseline is preserved.

```
python3 scripts/inventory.py /path/to/ProveIt /path/to/census
python3 scripts/replay.py /path/to/ProveIt
```

The replay script checks the commit and toolchain, builds only the dependencies selected by the included Lean imports, compiles each demonstration, records return codes, and saves output. It does not apply any candidate patch automatically. The README provides the separate static-generator and patch-check commands. Building from an empty cache can require substantial dependency downloads; the saved proof-check logs are included for immediate inspection.

12.2 Adoption sequence

1. Take the proof-body-only recursion patch. Rebuild its direct users and retain the program-data equality checks.
2. Take the list-library patch and assignment naturality change. Keep wrapper signatures, witness counts and expression data unchanged.
3. Add the shared SOS and topological-invariant theorems in modules low enough in the import graph. Migrate a small number of clients before expanding usage.
4. Update the circuit source generator, regenerate both Steps modules, compile the finite certificates and replay downstream Registers/Recipe/Translation modules. Compare public types and literal data hashes.
5. Introduce scanner helpers and conditional padding existence. For the latter, check every caller because its interface changes.
6. Consolidate digit/carry and model-theoretic interfaces in separate changes. These require careful attention to bounds, cyclic semantics, definability and logical contexts.

Each stage should remain understandable on its own. It is preferable to land a small correct interface with a few clients than to combine proof shortening, program optimization, theorem generalization and a new mathematical endpoint in one patch.

13 Further questions worth investigating

1. **What is the best certificate granularity?** Compare a single power-label certificate, fixed-size chunks, and per-node certificates. Keep the actual circuit list fixed and measure kernel time, memory and source size together.
2. **Can one translation interface cover later circuit variants?** Test the commutative local relation and explicit input/output correspondence on both current families before designing a larger compiler framework.
3. **Which scanner properties are truly shared?** Fixed-state laws are immediately reusable; encoded-list simulations need a precise relation between row storage, raw zero states and semantic histories. Determine the smallest interface that serves all four scanners.
4. **Can degree proofs be mostly certificate driven?** Separate purely structural bounds from cancellation-sensitive normal forms and exact-degree witnesses. Measure how many manual degree chains remain after this classification.

5. **How many object-language endpoints share one completeness bridge?** Inventory empty versus nonempty contexts and definability requirements before choosing the generic statement. A bridge should make these assumptions more explicit.
6. **Where does build time actually go?** After the small proof replacements, profile the large generated and finite classification modules. This can establish whether evaluator representation or certificate sharding is worth the wider refactor.

Source and artifact guide

All repository links in this report use the pinned commit. Detailed supplementary source notes are included for the checker/representation family, circuit family and digit/carry family. They contain additional line references and integration cautions. The definitive source measurements are `data/inventory_summary.json`, `data/files.csv`, and `data/generated-steps-metrics.json`. The archive manifest records SHA-256 hashes of every delivered member except the manifest itself.

The report's recommendations derive from inspected source and the attached checks. Larger interfaces described as proposals have not been turned into a complete repository refactor. The distinction lets the small verified changes be used immediately while retaining a clear research and engineering agenda for the larger simplifications.